03 — imem.v Explained: The Instruction Memory

Oct 6, 2026 · @shiv

1. Overview

`imem.v` stores the program: 256 words of 16 bits. The PC is the address, and the instruction at that address comes out on `instr` in the same cycle. The test program is written directly inside this file, so Vivado needs no separate program file.

Item	Value
File	design_sources/imem.v (86 lines)
Module name	imem
Pipeline stage	Stage 1 – Instruction Fetch (IF)
Type of logic	Read-only memory with combinational read
Size	256 × 16 bits = 4 kbit
Calls (instantiates)	Nothing – leaf module
Called by	risc8_5stage_core.v, instance u_imem, lines 68–71

1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v           <== THIS FILE
    ├── control_unit.v
    ├── regfile.v
    ├── hazard_unit.v
    ├── alu.v
    └── dmem.v
```

1.2 Ports

Port	Direction	Width	Meaning	Connected to (in risc8_5stage_core.v)
addr	input	8	Address of the instruction	pc (program counter)
instr	output	16	Instruction stored at that address	if_instr → IF/ID register

1.3 Dependencies

Modules, functions or tasks called inside this file: none. It is a leaf module.

Used by (file)	How	Instance	Lines
risc8_5stage_core.v	Module instance	u_imem	68–71

Related file: tb_risc8_5stage.v contains the expected results of this program. If you change the program, change the expected values there too.

1.4 Verilog constructs used

Construct	Meaning in plain words
`reg [15:0] rom [0:255]`	Array of 256 words, 16 bits each
`initial begin ... end`	Runs once at the start; sets the memory contents
`16'h823C`	A 16-bit number written in hexadecimal
`rom[8'h00] = ...`	Store a value at one address of the array
`assign`	Continuous read of the array

2. Code walkthrough, part by part

2.1 File header (lines 1–35)

The header says the program is built into this file, so no `.hex` file is needed. Lines 24–31 are a short guide to writing new instructions: the bit layout, the opcode numbers, and a reminder to update the expected results in tb_risc8_5stage.v. Lines 33–34 note that Vivado turns the initial block into a ROM.

2.2 Module and ports (lines 37–42)

```
`timescale 1ns/1ps

module imem (
    input  wire [7:0]  addr,    // = PC (from risc8_5stage_core.v)
    output wire [15:0] instr    // instruction stored at that address
);
```

There are only two ports: the 8-bit address (the PC) and the 16-bit instruction. There is no clock, because reading is combinational. The instruction appears in the same cycle as the address, so the IF stage finishes in one cycle.

2.3 The ROM array (lines 44–45)

```
    reg [15:0] rom [0:255];     // 256 instructions, 16 bits each
    integer i;
```

256 words of 16 bits: room for 256 instructions, which matches the 8-bit PC. `integer i` is the loop counter used on line 49.

2.4 Fill with NOP (lines 47–49)

```
    initial begin
        // Fill every location with NOP (0x0000) first.
        for (i = 0; i < 256; i = i + 1) rom[i] = 16'h0000;
```

An `initial` block runs once at time 0 in simulation. For an FPGA, Vivado uses it to set the memory's starting contents. All 256 words are first set to `16'h0000`, which is NOP. If the PC ever runs past the program, nothing harmful happens.

2.5 The test program (lines 51–81)

```
        // Part 1: ALU operations, forwarding, taken branch + flush
        rom[8'h00] = 16'h823C;   // LDI  R1, 0x3C      R1 = 0x3C
        rom[8'h01] = 16'h840F;   // LDI  R2, 0x0F      R2 = 0x0F
        rom[8'h02] = 16'h3650;   // AND  R3, R1, R2    R3 = 0x0C
        rom[8'h03] = 16'h4850;   // OR   R4, R1, R2    R4 = 0x3F
        rom[8'h04] = 16'h5AE0;   // XOR  R5, R3, R4    R5 = 0x33
        rom[8'h05] = 16'h6D40;   // SHL  R6, R5        R6 = 0x66
        rom[8'h06] = 16'h7F80;   // SHR  R7, R6        R7 = 0x33
        rom[8'h07] = 16'h2FE8;   // SUB  R7, R7, R5    R7 = 0x00, Z = 1
        rom[8'h08] = 16'hD00A;   // BZ   0x0A          taken (Z = 1)
        rom[8'h09] = 16'h8EFF;   // LDI  R7, 0xFF      wrong path: flushed
        // Part 2: loop, R2 = 5+4+3+2+1 = 15
        rom[8'h0A] = 16'h8205;   // LDI  R1, 5         loop counter
        rom[8'h0B] = 16'h8400;   // LDI  R2, 0         sum
        rom[8'h0C] = 16'h8601;   // LDI  R3, 1         constant 1
        rom[8'h0D] = 16'h1488;   // loop: ADD R2, R2, R1
        rom[8'h0E] = 16'h2258;   // SUB  R1, R1, R3    counter - 1
        rom[8'h0F] = 16'hE00D;   // BNZ  0x0D          repeat while not 0
        // Part 3: memory, load-use stall, carry, jump, halt
        rom[8'h10] = 16'h8820;   // LDI  R4, 0x20      address 0x20
        rom[8'h11] = 16'hB500;   // ST   R2, [R4]      MEM[0x20] = 15
        rom[8'h12] = 16'hAB00;   // LD   R5, [R4]      R5 = 15
        rom[8'h13] = 16'h9AF0;   // ADDI R5, 0xF0      R5 = 0xFF (stall +
forward)
        rom[8'h14] = 16'h9A01;   // ADDI R5, 1         R5 = 0x00, C = 1, Z =
1
        rom[8'h15] = 16'hC017;   // JMP  0x17          always taken
        rom[8'h16] = 16'h8CEE;   // LDI  R6, 0xEE      wrong path: flushed
        rom[8'h17] = 16'hF000;   // HLT                stop
        rom[8'h18] = 16'h8CAA;   // LDI  R6, 0xAA      never executed
    end
```

Each line stores one instruction. `rom[8'h00] = 16'h823C;` means "address 0x00 holds 0x823C". The comment on each line gives the assembly instruction and its effect.

The program has three parts: ALU operations with forwarding and a taken branch; a loop that adds 5+4+3+2+1; and memory access, a load-use stall, a carry, a jump and HLT. Addresses 0x09 and 0x16 are **wrong-path** instructions. They are fetched but must be flushed, and the testbench checks that they never change a register.

2.6 Reading (lines 83–86)

```
    // Asynchronous read: the instruction is available in the same cycle.
    assign instr = rom[addr];

endmodule
```

`assign` continuously outputs the word at the current address. `endmodule` closes the module.

3. The program decoded, writing new instructions, mistakes and viva

3.1 Every instruction split into its fields

Format: opcode [15:12] | rd [11:9] | rs1 [8:6] | rs2 [5:3] | 000. For LDI, ADDI, JMP, BZ and BNZ, bits [7:0] hold the 8-bit number.

Addr	Hex	Assembly	opcode	rd	Other fields	Effect
00	823C	LDI R1, 0x3C	1000	001	imm 0x3C	R1 = 0x3C
01	840F	LDI R2, 0x0F	1000	010	imm 0x0F	R2 = 0x0F
02	3650	AND R3, R1, R2	0011	011	rs1 001, rs2 010	R3 = 0x0C
03	4850	OR R4, R1, R2	0100	100	rs1 001, rs2 010	R4 = 0x3F
04	5AE0	XOR R5, R3, R4	0101	101	rs1 011, rs2 100	R5 = 0x33
05	6D40	SHL R6, R5	0110	110	rs1 101	R6 = 0x66
06	7F80	SHR R7, R6	0111	111	rs1 110	R7 = 0x33
07	2FE8	SUB R7, R7, R5	0010	111	rs1 111, rs2 101	R7 = 0, Z = 1
08	D00A	BZ 0x0A	1101	000	addr 0x0A	Taken
09	8EFF	LDI R7, 0xFF	1000	111	imm 0xFF	Flushed, never runs
0A	8205	LDI R1, 5	1000	001	imm 0x05	Loop counter
0B	8400	LDI R2, 0	1000	010	imm 0x00	Sum
0C	8601	LDI R3, 1	1000	011	imm 0x01	Constant 1
0D	1488	ADD R2, R2, R1	0001	010	rs1 010, rs2 001	Loop: sum += counter
0E	2258	SUB R1, R1, R3	0010	001	rs1 001, rs2 011	Counter − 1
0F	E00D	BNZ 0x0D	1110	000	addr 0x0D	Taken 4 times

Addr	Hex	Assembly	opcode	rd	Other fields	Effect
10	8820	LDI R4, 0x20	1000	100	imm 0x20	Address 0x20
11	B500	ST R2, [R4]	1011	010	rs1 100	MEM[0x20] = 15
12	AB00	LD R5, [R4]	1010	101	rs1 100	R5 = 15
13	9AF0	ADDI R5, 0xF0	1001	101	imm 0xF0	R5 = 0xFF (after 1 stall)
14	9A01	ADDI R5, 1	1001	101	imm 0x01	R5 = 0, C = 1, Z = 1
15	C017	JMP 0x17	1100	000	addr 0x17	Always taken
16	8CEE	LDI R6, 0xEE	1000	110	imm 0xEE	Flushed, never runs
17	F000	HLT	1111	000	–	Stop
18	8CAA	LDI R6, 0xAA	1000	110	imm 0xAA	Never executed

3.2 How to add your own instruction (example: SUB R4, R2, R1)

1. The opcode of SUB is 2 = **0010**.
2. rd = R4 = **100**, rs1 = R2 = **010**, rs2 = R1 = **001**, and the last three bits are **000**.
3. Join them: 0010 100 010 001 000 = 0010 1000 1000 1000 = **0x2888**.
4. Write `rom[8'h19] = 16'h2888;` and update the expected values in tb_risc8_5stage.v.

Branch targets (BZ, BNZ, JMP) are absolute addresses. If you insert an instruction, every target after that point must be updated.

3.3 Common mistakes

- Changing the program without updating the expected values in tb_risc8_5stage.v. The test then reports FAIL even though the CPU is correct.
- Forgetting that branch targets are absolute addresses.
- Putting the fields in the wrong order. Always build the word as opcode, rd, rs1, rs2, 000.

3.4 Viva questions

Q1. Why is the instruction memory read without a clock? So the instruction is available in the same cycle as the PC, and IF completes in one cycle.

Q2. What happens if the PC goes past the program? Every unused word is 0x0000, which is NOP, so nothing changes.

Q3. Why is the program inside the file instead of in a .hex file? Vivado often cannot find an external file read with $readmemh. Building the program in avoids that problem.

Q4. What hardware does Vivado build from this file? A 256 × 16 ROM, made from LUTs or a block RAM.